Advanced Rust (2026): Libraries and Testing

Lecture 4

Lukáš Hozda

Spring 2026

MFF CUNI

General Principles

Writing Libraries

- A library is judged mainly by its interface and long-term behavior

Writing Libraries

- A library is judged mainly by its interface and long-term behavior
- Internals matter, but callers mostly see consistency, reliability, and documentation

Writing Libraries

- A library is judged mainly by its interface and long-term behavior
- Internals matter, but callers mostly see consistency, reliability, and documentation
- Bad API decisions survive much longer than bad internal code

Consistency

- Similar things should be done the same way

Consistency

- Similar things should be done the same way
- Naming, ownership style, error handling, and mutability conventions should line up

Consistency

- Similar things should be done the same way
- Naming, ownership style, error handling, and mutability conventions should line up
- Inconsistency forces users to memorize special cases

Consistency Example

- Suppose we expose a `Fraction` type

Consistency Example

- Suppose we expose a `Fraction` type
- Bad design: `add` and `subtract` return new values, but `multiply` and `divide` mutate in place

Consistency Example

- Suppose we expose a `Fraction` type
- Bad design: `add` and `subtract` return new values, but `multiply` and `divide` mutate in place
- Good design: all arithmetic operations follow one coherent model

Consistency Example

- Suppose we expose a `Fraction` type
- Bad design: `add` and `subtract` return new values, but `multiply` and `divide` mutate in place
- Good design: all arithmetic operations follow one coherent model
- In Rust, users expect behavior to resemble primitive operators where possible

API Stability

- Public types, traits, functions, constants, and modules form the API

API Stability

- Public types, traits, functions, constants, and modules form the API
- Internal rewrites are cheap only if the public surface is narrow enough

API Stability

- Public types, traits, functions, constants, and modules form the API
- Internal rewrites are cheap only if the public surface is narrow enough
- Good library design is partly about leaving room for future change

SemVer

- Cargo understands semantic versioning

SemVer

- Cargo understands semantic versioning
- In practice, users rely on compatible upgrades not breaking their code

SemVer

- Cargo understands semantic versioning
- In practice, users rely on compatible upgrades not breaking their code
- Breaking changes are sometimes necessary, but they should not be casual

Encapsulation

- Hide internal representation unless exposing it is truly necessary

Encapsulation

- Hide internal representation unless exposing it is truly necessary
- Prefer private fields and explicit constructors or methods

Encapsulation

- Hide internal representation unless exposing it is truly necessary
- Prefer private fields and explicit constructors or methods
- Hidden representation improves safety, refactoring freedom, and invariants

Ease of Use

- Library should be easy to start with

Ease of Use

- Library should be easy to start with
- Library should also be hard to misuse

Ease of Use

- Library should be easy to start with
- Library should also be hard to misuse
- Good defaults, clear types, and small API surface help more than slogans about ergonomics

Rust Library Design

Rust Style

- Idiomatic Rust is usually the path of least resistance once the design is sound

Rust Style

- Idiomatic Rust is usually the path of least resistance once the design is sound
- Rust pushes strongly toward explicit ownership, small interfaces, and trait-based reuse

Rust Style

- Idiomatic Rust is usually the path of least resistance once the design is sound
- Rust pushes strongly toward explicit ownership, small interfaces, and trait-based reuse
- Fighting the language is usually a sign of design mismatch

Documentation and Tests

- In Rust, documentation examples can be executable tests

Documentation and Tests

- In Rust, documentation examples can be executable tests
- This is useful because it checks that the documented usage still compiles and runs

Documentation and Tests

- In Rust, documentation examples can be executable tests
- This is useful because it checks that the documented usage still compiles and runs
- Documentation quality directly affects API quality

Doc Tests

```
/// # Examples
/// let mut vec = Vec::with_capacity(3);
/// vec.push(1);
/// vec.push(2);
/// assert_eq!(vec.len(), 2);
```

Doc-Test Notes

- `cargo test` runs doctests too

Doc-Test Notes

- `cargo test` runs doctests too
- Use # to hide support lines in examples

Doc-Test Notes

- `cargo test` runs doctests too
- Use `#` to hide support lines in examples
- Use `no_run` when the code should compile but not execute

Doc-Test Notes

- `cargo test` runs doctests too
- Use `#` to hide support lines in examples
- Use `no_run` when the code should compile but not execute
- `#![deny(missing_docs)]` is reasonable for real libraries

Standard Crate Layout

```
.
|-- Cargo.toml
|-- benches
|-- examples
|-- src
|   |-- bin
|   |-- lib.rs
|   `-- main.rs
`-- tests
```

Make Illegal States Unrepresentable

- Type system should rule out nonsense where practical

Make Illegal States Unrepresentable

- Type system should rule out nonsense where practical
- This is one of Rust's biggest strengths

Make Illegal States Unrepresentable

- Type system should rule out nonsense where practical
- This is one of Rust's biggest strengths
- If invalid states are easy to construct, the API is weak

Stringly Typed API

```rust
use chrono::{DateTime, Utc, Weekday};

fn is_matching_day(datetime: DateTime<Utc>, day: &str) -> bool {
    let weekday = datetime.weekday();
    match day.to_lowercase().as_str() {
        "monday" => weekday == Weekday::Mon,
        "tuesday" => weekday == Weekday::Tue,
        "wednesday" => weekday == Weekday::Wed,
```

```
        "thursday" => weekday == Weekday::Thu,
        "friday" => weekday == Weekday::Fri,
        "saturday" => weekday == Weekday::Sat,
        "sunday" => weekday == Weekday::Sun,
        _ => unreachable!("there are only seven days in a week"),
    }
}
```

Why This Is Bad

```
let is_tuesday = is_matching_day(
    some_date,
    "If you ask Rick Astley for a copy of the movie UP, he cannot give you it",
);
```

- Random strings are the wrong representation of a finite domain

Why This Is Bad

```
let is_tuesday = is_matching_day(
    some_date,
    "If you ask Rick Astley for a copy of the movie UP, he cannot give you it",
);
```

- Random strings are the wrong representation of a finite domain
- The fix is not better validation text, but a better type

Why This Is Bad

```
let is_tuesday = is_matching_day(
    some_date,
    "If you ask Rick Astley for a copy of the movie UP, he cannot give you it",
);
```

- Random strings are the wrong representation of a finite domain
- The fix is not better validation text, but a better type
- Enum is the natural model here

Builder Pattern

- Rust has neither default parameters nor variadic functions in the C++ sense

Builder Pattern

- Rust has neither default parameters nor variadic functions in the C++ sense
- Builder pattern is common for large configurable constructors

Builder Pattern

- Rust has neither default parameters nor variadic functions in the C++ sense
- Builder pattern is common for large configurable constructors
- It also hides internal representation and allows validation at `build`

Builder Example

```
fn main() {
    let car = CarBuilder::new("Toyota", "Corolla")
        .year(2020)
        .color(Color::Blue)
        .automatic(true)
        .build();

    println!("{:#?}", car);
}
```

Builder Notes

- Typical shape: `new` then chained setters then `build`

Builder Notes

- Typical shape: `new` then chained setters then `build`
- Builder methods usually return `Self` or `&mut Self`

Builder Notes

- Typical shape: `new` then chained setters then `build`
- Builder methods usually return `Self` or `&mut Self`
- Session-type builders can enforce ordering constraints through different intermediate types

Conversion and Trait Design

Accept Flexible Input Types

```rust
use std::fs;
use std::path::Path;

fn file_exists(path: &Path) -> bool {
    fs::metadata(path).is_ok()
}
```

- Works, but it forces callers into one concrete representation

AsRef<Path>

```rust
use std::fs;
use std::path::Path;

fn file_exists<P: AsRef<Path>>(path: P) -> bool {
    fs::metadata(path.as_ref()).is_ok()
}
```

```rust
fn main() {
    let path_str = "./example.txt";
    println!("{}", file_exists(path_str));

    let path_buf = Path::new("./example.txt").to_path_buf();
    println!("{}", file_exists(path_buf));
}
```

Useful Conversion Traits

- `AsRef` and `AsMut` for cheap borrowed conversion

Useful Conversion Traits

- `AsRef` and `AsMut` for cheap borrowed conversion
- `From` and therefore `Into` for value conversion

Useful Conversion Traits

- `AsRef` and `AsMut` for cheap borrowed conversion
- `From` and therefore `Into` for value conversion
- `TryFrom` and `TryInto` for fallible conversion

Useful Conversion Traits

- `AsRef` and `AsMut` for cheap borrowed conversion
- `From` and therefore `Into` for value conversion
- `TryFrom` and `TryInto` for fallible conversion
- `FromStr` to support `.parse()` on strings

Useful Traits to Implement

- `Debug` and often `Display`

Useful Traits to Implement

- `Debug` and often `Display`
- `Display` and `Error` for error types

Useful Traits to Implement

- `Debug` and often `Display`
- `Display` and `Error` for error types
- `Clone`, sometimes `Copy`

Useful Traits to Implement

- `Debug` and often `Display`
- `Display` and `Error` for error types
- `Clone`, sometimes `Copy`
- `PartialEq`, `Eq`, `PartialOrd`, `Ord`, `Hash` where meaningful

Useful Traits to Implement

- `Debug` and often `Display`
- `Display` and `Error` for error types
- `Clone`, sometimes `Copy`
- `PartialEq`, `Eq`, `PartialOrd`, `Ord`, `Hash` where meaningful
- `Default`, iterator traits, and serde traits where appropriate

FromIterator

```
struct MyCollection<T> {
    elements: Vec<T>,
}

impl<T> FromIterator<T> for MyCollection<T> {
    fn from_iter<I: IntoIterator<Item = T>>(iter: I) -> Self {
        let mut c = MyCollection { elements: Vec::new() };
```

```
        for i in iter {
            c.elements.push(i);
        }
        c
    }
}

let collected: MyCollection<i32> = vec![1, 2, 3, 4, 5].into_iter().collect();
```

Orphan Rules

- You cannot implement a foreign trait for a foreign type

Orphan Rules

- You cannot implement a foreign trait for a foreign type
- This is Rust's coherence rule at work

Orphan Rules

- You cannot implement a foreign trait for a foreign type
- This is Rust's coherence rule at work
- The usual escape hatch is a newtype wrapper

Newtype Example

```rust
use std::fmt;

pub struct DisplayVec<T>(pub Vec<T>);

impl<T: fmt::Display> fmt::Display for DisplayVec<T> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let xs: Vec<String> = self.0.iter().map(|e| e.to_string()).collect();
        write!(f, "[{}]", xs.join(", "))
    }
}
```

Extension Traits

- Extension traits add methods to existing types in a controlled way

Extension Traits

- Extension traits add methods to existing types in a controlled way
- Conventionally named `SomethingExt`

Extension Traits

- Extension traits add methods to existing types in a controlled way
- Conventionally named `SomethingExt`
- They are useful when you own the trait but not the target type

Testing

Test Categories

- Unit tests sit next to the code and may inspect internals

Test Categories

- Unit tests sit next to the code and may inspect internals
- Integration tests live in `tests/` and treat the crate as an external dependency

Test Categories

- Unit tests sit next to the code and may inspect internals
- Integration tests live in `tests/` and treat the crate as an external dependency
- Benchmarks are a different concern again

Test Syntax

```rust
#[cfg(test)]
mod tests {
    #[test]
    fn test_addition() {
        let sum = 2 + 2;
        assert_eq!(sum, 4);
    }
```

```
    #[test]
    #[should_panic(expected = "assertion failed")]
    fn test_failure_scenario() {
        assert!(false, "This test will panic!");
    }
}
```

Result-Returning Tests

```rust
#[cfg(test)]
mod tests {
    #[test]
    fn test_division() -> Result<(), String> {
        let result = 10 / 2;
        if result == 5 {
            Ok(())
```

```
        } else {
            Err(String::from("Division result was not as expected"))
        }
    }
}
```

Dev Dependencies

- Put test-only and benchmark-only crates into `[dev-dependencies]`

Dev Dependencies

- Put test-only and benchmark-only crates into `[dev-dependencies]`
- Typical examples: `criterion`, proptest, snapshot testing tools

Dev Dependencies

- Put test-only and benchmark-only crates into `[dev-dependencies]`
- Typical examples: `criterion`, proptest, snapshot testing tools
- Keep library dependencies and test dependencies conceptually separate

Benchmarking

```rust
use criterion::{black_box, criterion_group, criterion_main, Criterion};

fn fibonacci(n: u64) -> u64 {
    match n {
        0 => 0,
        1 => 1,
        _ => fibonacci(n - 1) + fibonacci(n - 2),
    }
}
```

```rust
fn criterion_benchmark(c: &mut Criterion) {
    c.bench_function("fibonacci 20", |b| b.iter(|| fibonacci(black_box(20))));
}

criterion_group!(benches, criterion_benchmark);
criterion_main!(benches);
```

Maintenance

Maintenance Tools

- `rustfmt` for formatting

Maintenance Tools

- `rustfmt` for formatting
- `clippy` for linting

Maintenance Tools

- `rustfmt` for formatting
- `clippy` for linting
- CI should run both automatically

Maintenance Tools

- `rustfmt` for formatting
- `clippy` for linting
- CI should run both automatically
- Publishing code with warnings is usually a bad sign

Warnings and Checks

- `#![deny(warnings)]` is common in library code

Warnings and Checks

- `#![deny(warnings)]` is common in library code
- You can enable additional compiler and clippy lints

Warnings and Checks

- `#![deny(warnings)]` is common in library code
- You can enable additional compiler and clippy lints
- Other useful tools: `cargo-machete`, `cargo-outdated`, `cargo-llvm-cov`, `cargo-tarpaulin`

Summary

- Keep API small, consistent, and type-driven

Summary

- Keep API small, consistent, and type-driven
- Integrate with standard traits where it helps

Summary

- Keep API small, consistent, and type-driven
- Integrate with standard traits where it helps
- Write tests and documentation as part of the design process

Summary

- Keep API small, consistent, and type-driven
- Integrate with standard traits where it helps
- Write tests and documentation as part of the design process
- Use tooling so maintenance is not left to memory

Questions?